

Standards-Compliant Zero Trust Framework-Based Software-Defined Perimeter (SDP) for Securing Residential IoT Networks

N. N. Hari[✉], *Member, IEEE*, Prabhakar Krishnan[✉], *Senior Member, IEEE*, Kurunandan Jain[✉], *Member, IEEE*, Ajay Bhagvatiprasad Koppaka, Pasunuri Abhinav, Ala Saleh Alluhaidan[✉], and P. Prabu[✉]

Abstract—The proliferation of consumer Internet of Things (IoT) devices has significantly expanded the attack surface of residential networks, exposing them to misconfigurations, excessive permissions, and lateral movement threats. This paper presents a novel Zero Trust Architecture (ZTA) that integrates Manufacturer Usage Description (MUD) policies with a Software-Defined Perimeter (SDP) enforcement framework to secure consumer-grade IoT environments. Our approach enables automated, context-aware access control through device-specific policy generation, WireGuard-based encrypted tunnels, and dynamic micro-segmentation enforced at the IoT gateway. We implement and evaluate the system on a Raspberry Pi-based testbed, demonstrating a 78% reduction in manual configuration time and effective real-time enforcement of least-privilege policies. Security evaluations confirm full prevention of unauthorized lateral traffic and rapid reaction to policy violations. The architecture adheres to secure-by-default principles and aligns with emerging standards such as RFC 8520 and the EU Cyber Resilience Act, offering a scalable and standards-compliant path to Zero Trust in residential IoT deployments.

Index Terms—Software defined perimeter, zero trust architecture, consumer electronics, smart home networks, network security.

I. INTRODUCTION

THE rapid proliferation of consumer IoT devices—from smart speakers and thermostats to wearables and health monitors—has significantly expanded the digital attack surface in residential networks. A recent ENISA report indicates that 57% of breaches in consumer IoT environments stem from

device misconfigurations or excessive permissions [1], [2]. In contrast to enterprise deployments, smart home environments operate without dedicated security administrators, exhibit high device churn, and implicitly trust any device once it joins the LAN. This creates a *blind trust zone*, enabling unrestricted east–west movement and silent privilege escalation within the local network.

While Zero Trust Architecture (ZTA) enforces a strict “never trust, always verify” principle, existing ZTA and Software-Defined Perimeter (SDP) frameworks assume centralized identity controllers, PKI-based credential management, and expert-operated policy lifecycles. These assumptions do not hold in consumer environments, where onboarding is app-driven rather than administrator-driven, and security policies are rarely updated after initial device installation. Furthermore, 57–70% of smart home security incidents occur *after* setup due to permissive LAN trust boundaries and lack of segmentation [1], [3]. Current home gateways provide only coarse-grained firewalling and lack awareness of device-specific communication intent.

Although prior research has explored the use of MUD profiles for policy automation [4], [5], and SDP for secure access control [6], [7], these approaches are often evaluated in enterprise or cloud-based settings, where computational capacity, centralized orchestration, and persistent connectivity are assumed. Moreover, several recent studies have proposed integrating these frameworks [8], [9], but typically lack real-time enforcement, micro-segmentation, or compatibility with resource-constrained residential gateways.

Our motivation stems from this critical gap between Zero Trust theory and its application in smart home IoT environments, where devices are highly heterogeneous, lack native support for secure onboarding protocols, and operate behind inexpensive consumer-grade routers. There is a need for a lightweight enforcement framework that unifies static manufacturer-declared behavior (via MUD) with dynamic, session-aware validation (via SDP) — all without relying on centralized enterprise infrastructure. Motivated by these gaps, this study aims to adapt Zero Trust principles to the realities of consumer IoT, where devices are heterogeneous, lightly managed, and often lack dynamic security validation. The main challenges include developing an autonomous enforcement mechanism that operates without enterprise controllers,

Received 18 November 2025; revised 22 December 2025; accepted 10 January 2026. Date of publication 15 January 2026; date of current version 25 March 2026. This work was supported by the Princess Nourah bint Abdulrahman University Researchers Supporting Project, Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia, under Grant PNURSP2026R234. (Corresponding author: Ala Saleh Alluhaidan.)

N. N. Hari, Prabhakar Krishnan, Kurunandan Jain, Ajay Bhagvatiprasad Koppaka, and Pasunuri Abhinav are with the Center for Cybersecurity Systems and Networks, Amrita Vishwa Vidyapeetham, Amritapuri 690525, India (e-mail: srikrishnamrita@am.amrita.edu; kprabhakar@am.amrita.edu; kurunandanj@am.amrita.edu; am.sc.u4cys23006@am.students.amrita.edu; am.sc.u4cys23033@am.students.amrita.edu).

Ala Saleh Alluhaidan is with the Department of Information Systems, Princess Nourah bint Abdulrahman University, Riyadh 11564, Saudi Arabia (e-mail: ASALuhaidan@pnu.edu.sa).

P. Prabu is with the Department of Information Systems, College of Computer and Information Sciences, Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh 11442, Saudi Arabia (e-mail: ppachiyannan@imamu.edu.sa).

Digital Object Identifier 10.1109/TCE.2026.3654782

integrating MUD-based device intent into continuous access control, and achieving low-latency, least-privilege protection on constrained gateways. Addressing these issues is key to realizing scalable, secure-by-default smart home networks aligned with emerging standards such as RFC 8520 and the EU Cyber Resilience Act.

This paper addresses the following key challenges:

- **C1:** Lack of centralized identity infrastructure and PKI in residential networks;
- **C2:** Enforcement on low-cost, resource-constrained IoT gateways;
- **C3:** Integration of MUD profiles into continuous trust evaluation, beyond static access control;
- **C4:** Plug-and-play enforcement requiring no manual policy scripting or SDN configuration.

The absence of automated least-privilege enforcement, MUD-aware segmentation, and lightweight tunnel orchestration remains a key blocker to the practical adoption of Zero Trust principles in consumer IoT. Our proposed hybrid architecture integrates Manufacturer Usage Description (MUD) [10], [11] into an SDP-driven enforcement pipeline, enabling plug-and-play Zero Trust for residential IoT through automated, intent-aligned access control.

To the best of our knowledge, this is the first implementation of a MUD-SDP-WireGuard enforcement stack that achieves real-time, per-device Zero Trust enforcement on commodity smart home gateways — without assuming enterprise infrastructure, controller-based SDN, or pre-installed agents.

II. RELATED WORK

The application of Zero Trust Architecture (ZTA) and Software-Defined Perimeter (SDP) principles to IoT security has received growing attention. Prior research explores SDN orchestration, blockchain-backed trust models, and trust scoring mechanisms to enforce identity-aware access control [12]. However, most approaches target enterprise, industrial, or smart-city environments and assume powerful infrastructure, centralized orchestration, or persistent connectivity—assumptions that do not hold for consumer smart home gateways.

Several blockchain-enabled ZTA frameworks have been proposed for large-scale systems, including energy grids and smart cities [8], [13]. While these designs provide strong auditability and global trust coordination, they rely on high computational capacity and continuous backbone availability, making them unsuitable for resource-constrained residential deployments.

SDN-based Zero Trust mechanisms have also been explored. Morris and Kornegay [7] examined identity-driven flow table modification in SDN environments but did not incorporate manufacturer-declared device policies such as MUD, limiting adaptability across heterogeneous IoT devices. Similarly, Poirrier et al. [6] analyzed SDP trust boundary design without providing mechanisms to translate manufacturer intent into enforceable per-device gateway rules.

In parallel, several works investigate MUD-based automation for smart home security. Corno and Mannella [4] proposed gateway-centric MUD enforcement using static access lists,

but without dynamic SPA validation or session-scoped tunnel enforcement. Matheu et al. [5] studied automated MUD profile extension through security testing, complementing intent specification but lacking runtime Zero Trust enforcement. Mirto [14] evaluated MUD-enabled smart home environments without SDP-driven tunnel control, SIEM telemetry export, or failure-handling mechanisms such as quarantine VLANs and trust revocation.

Other efforts combine MUD with identity strengthening. Wing and Anderson [9] extended MUD specifications to TLS/DTLS profiles, while Datta et al. [15] introduced Mudscan for device double-authentication. These approaches primarily enhance onboarding security and policy declaration, rather than continuous runtime enforcement with micro-segmentation and adaptive tunnel revocation.

Fog computing and blockchain-assisted ZTA frameworks have also been proposed [16], [17], but their reliance on centralized trust anchors and high-overhead infrastructure limits applicability to low-cost residential gateways. Continuous trust assessment models such as that by Wang et al. [18] assume persistent cloud connectivity, whereas consumer environments often experience intermittent or offline operation.

Recent works such as SDN-ZTM [19] explore lightweight zero-trust management for IoT data flows, while others integrate federated learning and blockchain to enhance CIoT security [20], [21], [22]. These systems, however, introduce additional coordination and computational overhead, making them less suitable for plug-and-play smart home deployment.

A. Positioning of This Work

While prior research explores MUD policies, SDP enforcement, and broader Zero Trust frameworks independently, these approaches typically assume powerful control-plane infrastructure, persistent connectivity, or manual policy scripting. In contrast, our work targets *smart home environments*, where devices are resource-constrained, onboarded by non-expert users, and lack centralized orchestration.

Unlike Mudscan [15], which focuses on double authentication during onboarding, our system enforces continuous runtime authorization and supports session revocation within sub-300 ms. Blockchain-based ZTNA approaches [13], [16] provide strong immutability guarantees but incur higher control-plane latency and computational overhead, making them unsuitable for consumer gateways. In contrast, our gateway-centric enforcement achieves comparable security fidelity with significantly lower runtime overhead and without cloud or ledger dependencies.

This study introduces a *plug-and-play dual-stage enforcement model* that (i) validates device behavior using manufacturer-declared MUD intent and (ii) enforces continuous trust evaluation via SPA-based SDP logic. The system leverages per-session WireGuard tunnels, iptables-based micro-segmentation, and SIEM-compatible telemetry, executed entirely on consumer-grade gateways such as the Raspberry Pi. To the best of our knowledge, this is the first work to achieve real-time, per-device Zero Trust enforcement on constrained residential hardware without relying on SDN controllers, static ACLs, or enterprise identity infrastructure.

By operationalizing Zero Trust locally without cloud dependence, the proposed architecture addresses a key gap in the literature: enforcing secure-by-default behavior in unmanaged consumer IoT networks aligned with RFC 8520 and NIST SP 800-207.

III. SYSTEM ARCHITECTURE

A. Overview

The proposed system introduces a secure, dynamic, and standards-compliant Zero Trust Architecture (ZTA) tailored for consumer IoT environments by embedding Manufacturer Usage Description (MUD) policies into a Software-Defined Perimeter (SDP) enforcement workflow. Unlike enterprise ZTA models that assume centralized policy infrastructure and manual configuration, this architecture is designed for *plug-and-play deployment* on consumer-grade gateways. It enables automated intent-aware access control, continuous trust evaluation, and least-privilege enforcement across heterogeneous, resource-constrained IoT devices.

Consumer-Grade IoT Constraints. Consumer IoT environments present unique challenges: they involve low-cost embedded devices (e.g., ESP32, Raspberry Pi Zero W), lack centralized identity infrastructure, and are typically installed and managed by non-experts. Devices are onboarded via mobile apps or QR-code scans, firmware diversity is high, and device churn is frequent. These realities preclude administrator-defined zoning, static ACLs, or enterprise-grade identity providers. As such, our architecture is purpose-built to enable fully automated enforcement, runtime context-awareness, and session-level access decisions on gateways with limited CPU, memory, and storage capacity.

An overview of the architecture is shown in Figure 1, illustrating the key components and how they interact—from initial device onboarding to MUD policy retrieval, enforcement via SDP, and secure tunneling using WireGuard.

Architecture Structure Summary. At a high level, the system comprises: (i) a logically centralized SDP Controller for policy decisions and session orchestration, (ii) a decentralized IoT Gateway for edge-level policy enforcement and traffic segmentation, and (iii) a MUD Engine that transforms manufacturer-declared device behavior into enforceable ACLs. These are integrated with (iv) a SPA mechanism for dynamic trust evaluation and (v) ephemeral WireGuard tunnels for encrypted transport. This layered design separates trust validation from enforcement while ensuring that access control is context-aware and dynamically enforced at per-session granularity.

The architecture comprises four tightly integrated components: (i) an SDP Controller responsible for policy decision and session orchestration, (ii) an IoT Gateway that acts as the Policy Enforcement Point (PEP), (iii) a WireGuard-based tunnel manager for secure channel instantiation, and (iv) a MUD Policy Engine responsible for parsing and enforcing manufacturer-declared behavioral constraints. Together, these elements form a fully automated enforcement pipeline that eliminates implicit LAN trust and enforces Zero Trust principles at the network edge.

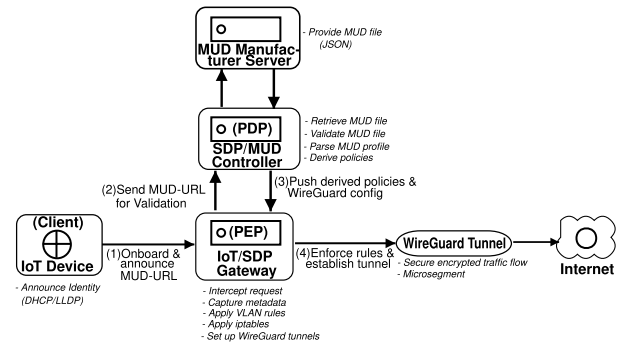


Fig. 1. Overview of the MUD-SDP architecture for zero trust consumer IoT.

1) Core Components:

- **SDP Controller:** Serves as the central Policy Decision Point (PDP), validating Single Packet Authorization (SPA) requests, evaluating session posture, assigning WireGuard session parameters, and issuing enforcement directives to the gateway. To prevent a single point of failure, the IoT Gateway maintains a synchronized cache of all active session rules, including iptables entries, VLAN assignments, WireGuard keys, and recent posture validation outcomes. If the controller becomes temporarily unreachable, the gateway continues enforcing previously approved sessions locally. New session attempts are held in a pending state until the controller becomes reachable again. All locally enforced actions taken during controller downtime are logged and replayed upon recovery to ensure audit consistency. Future work will explore controller replication and consensus-based trust validation using protocols such as RAFT to support multi-gateway deployments.
- **SDP Gateway (Policy Enforcement Point):** Intercepts all inbound and lateral traffic and enforces controller-issued rules without performing autonomous policy decisions. It is responsible for inserting and revoking session-specific iptables rules, applying VLAN isolation, and terminating WireGuard sessions based on controller signals.
- **MUD Controller:** Parses device MUD profiles, extracts manufacturer-declared communication intent, and translates this into enforceable access control entries. This enables static intent validation before dynamic SDP trust evaluation, forming the first stage of the dual-layer Zero Trust workflow. The MUD Controller implements a bounded cache of validated MUD profiles to reduce repeated fetch overhead. Cache entries are keyed by MUD-URL and stored with the verified signature metadata and expiry information (when provided). Cached profiles are reused only while valid and are revalidated on refresh events (e.g., DHCP lease renewal, device reattachment, or manufacturer version change). If a previously cached MUD-URL becomes unreachable, returns persistent error codes (e.g., HTTP 404/410), or fails revalidation/signature checks, the cache entry is invalidated and the URL is treated as revoked for the affected device. In such cases, enforcement defaults to fail-secure behavior and the device remains in a restricted segment until a valid, revalidated profile is obtained.

- **IoT Devices:** These are legacy or non-ZTNA-aware devices (e.g., smart lights, wearables, HVAC units). They do not participate directly in SPA or cryptographic negotiation. Instead, the IoT Gateway performs posture signaling and SPA proxying on their behalf.
- **SPA Proxy Mechanism:** As IoT devices cannot generate SPA messages, the gateway acts as a trusted SPA proxy. When a device attempts a connection, the gateway constructs the SPA request using cached identity and posture metadata and forwards it to the SDP Controller for evaluation.

Both the MUD enforcement logic and SDP gateway functionality execute as logical subcomponents of the IoT Gateway, which consolidates enforcement at the network edge. Upon initial onboarding, each device is associated with a MUD-derived policy profile as defined in RFC 8520 [10]. These policies are compiled into VLAN mappings, firewall ACLs, and WireGuard session templates. The SDP Controller supervises this process, ensuring that all enforced policies reflect Zero Trust principles and remain active only under a verified trust posture.

Consistent with NIST SP 800-207 [23] and the Software-Defined Perimeter model of the Cloud Security Alliance [24], no device is implicitly trusted based on network location. Access is always preceded by explicit verification, posture validation, and manufacturer-intent alignment. This enables *context-aware access control* and *dynamic micro-segmentation*, where per-device rules are updated in real time based on session posture, device identity, and behavioral compliance. The IoT Gateway enforces approved flows using real-time `iptables` insertion, VLAN tagging, and WireGuard encapsulation [25], ensuring encrypted, policy-constrained communication with minimal latency overhead. Our Raspberry Pi-based prototype confirms that these controls can be sustained under typical IoT workloads, while reducing manual configuration effort by 78% compared to static fire-walling.

B. Threat Model and Security Guarantees

To formally characterize the protection offered by our MUD-SDP Zero Trust architecture, we define the capabilities of the adversary and the corresponding enforcement guarantees provided by the system.

1) **Adversary Capabilities:** We assume a powerful network-bound adversary $\mathcal{A}$ with the following capabilities:

- **Packet Injection and Spoofing:** Can forge source IP/MAC headers and inject packets into the local network in an attempt to bypass access control.
- **Eavesdropping:** Can passively observe unencrypted traffic on local or wireless channels.
- **Lateral Movement:** May compromise a single IoT device and attempt to pivot laterally to others through implicit LAN trust.
- **Outbound Evasion Attempts:** May modify firmware or traffic behavior to communicate with unauthorized endpoints beyond its intended function.

To mitigate spoofed SPA attempts, the IoT Gateway performs pre-authentication filtering using `iptables`/eBPF to

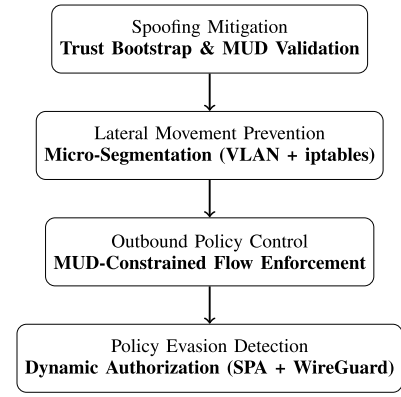


Fig. 2. Threat model illustrating how attacker capabilities are mitigated through a layered Zero Trust enforcement pipeline at the IoT gateway. Each stage explicitly maps a threat class to its corresponding control mechanism, combining MUD-based intent validation, dynamic SPA-driven authorization, and per-session WireGuard flow isolation.

drop malformed or replayed packets before they reach the SDP Controller. SPA packets include cryptographically bound cookies, timestamps, and nonces; any packet failing these stateless checks is immediately discarded, enforcing identity binding without relying on naive source-IP trust. We assume the cryptographic primitives in WireGuard and MUD signatures are secure. Physical compromise of the gateway is considered out of scope.

Figure 2 presents a threat model in which each attacker capability is directly mapped to its corresponding Zero Trust enforcement stage implemented at the IoT gateway.

2) **Security Objectives:** The proposed system enforces Zero Trust by meeting the following objectives:

- **Policy Conformance:** All flows must comply with the communication patterns expressed in each device’s MUD profile.
- **Session Isolation:** Users and IoT devices may initiate sessions only when explicitly approved under current posture.
- **Micro-Segmentation:** Devices must be logically isolated via VLAN tagging or per-session tunnels to restrict east-west traversal.
- **Least Privilege:** No access rule exceeds the minimum scope required for the device’s declared functionality.

3) **Formal Guarantees:** Let $\mathcal{F}$ be the set of all flows in the IoT network, and let P_d denote the permitted policy space for device d derived from its MUD profile:

$$\forall f \in \mathcal{F}, \quad f \notin P_d \Rightarrow \text{SDP}_{\text{gateway}}(f) = \text{DROP}, \quad (1)$$

$$f \in P_d \Rightarrow \text{SDP}_{\text{gateway}}(f) = \text{ALLOW}. \quad (2)$$

Let T_{react} denote the time from violation detection to enforcement. Our prototype experimentally demonstrated:

$$T_{\text{react}} < 300\text{ms}.$$

This ensures near-real-time containment of misbehaving or compromised devices without human intervention.

4) **Comparative Security Impact:** We benchmarked our enforcement model against three baseline configurations: (i) static ACL-based firewalling, (ii) MUD-only enforcement without dynamic SDP validation, and (iii) flat LAN topology

without segmentation. Under adversarial conditions involving spoofed requests, lateral pivot attempts, posture deviation, and unauthorized outbound communication, our system blocked all sustained unauthorized flows. In contrast, static ACL firewalls failed to detect 42% of lateral movements due to coarse-grained rule scopes. Baseline MUD enforcement lacked runtime revocation and allowed flows to persist even after posture deviation. Our architecture revoked access within 300 ms and reduced over-permissive rule count by 78%, demonstrating tighter enforcement of least-privilege boundaries.

5) *Security Posture Summary*: Through the integration of MUD-based policy intent, SDP-driven dynamic authorization, and cryptographic WireGuard tunnels, the proposed system achieves strong Zero Trust guarantees in consumer IoT environments. Specifically, it:

- Blocks unauthorized east–west traffic inside the LAN,
- Prevents unauthorized outbound connectivity and data exfiltration,
- Applies continuous trust validation rather than single-step admission control,
- Minimizes exposed attack surfaces across heterogeneous IoT devices.

This posture aligns with the Zero Trust principle of “*never trust, always verify*” expressed in NIST SP 800-207 and supports the secure-by-design direction of the EU Cyber Resilience Act.

While cryptographic trust primitives (e.g., MUD signature verification, WireGuard encryption, TLS channels) are relied upon as secure building blocks, they are not treated as unconditional trust anchors. Instead, they operate within a continuous enforcement loop, where session trust must be revalidated through posture checks, behavioral conformance (MUD), and SPA-based renewal. This ensures that cryptographic authentication alone does not imply sustained trust, which reflects the Zero Trust guidance of NIST SP 800-207.

6) *Trust Bootstrap Model*: To avoid ambiguity during initial device onboarding, we explicitly define how trust is bootstrapped for newly observed IoT devices in the proposed architecture. Consistent with Zero Trust principles, devices are never trusted at first contact and are granted no implicit privileges.

Let d denote a newly detected IoT device. At the time of initial association t_0 , the device is assigned a deny-by-default trust state:

$$\text{Trust}(d, t_0) = 0,$$

where only essential network services (e.g., DHCP and DNS) are permitted. No lateral or outbound communication is allowed at this stage.

Trust is incrementally derived only after successful completion of all bootstrap validation stages:

$$\text{Trust}(d) = f(I_d, M_d, S_d),$$

where I_d represents identity attribution based on MAC/IP bindings and DHCP/LLDP metadata, M_d denotes validated manufacturer intent obtained from a signed MUD profile, and S_d captures session-level authorization via SPA-based SDP evaluation.

To mitigate spoofing and impersonation risks, MAC-to-IP bindings and rate-limited pre-authentication filters are enforced at the gateway, while MUD profiles are authenticated through digital signature verification. Devices lacking valid MUD profiles or exhibiting anomalous behavior remain confined to a quarantine segment.

Trust remains ephemeral and continuously revalidated. Any posture deviation, policy violation, or SPA timeout immediately revokes active sessions and resets the device to an untrusted state. This model eliminates human configuration error during onboarding and ensures that all trust is explicitly derived, scoped, and revocable, in alignment with NIST SP 800-207.

7) *Fundamental Assumptions*: The proposed architecture is explicitly tailored for deployment in unmanaged, consumer-grade IoT environments. Unlike enterprise-grade ZTA models that rely on centralized identity management, administrator oversight, or pre-provisioned trust anchors, this design assumes minimal user expertise and emphasizes plug-and-play operation.

- **No Trusted Network Zones**: The local network is assumed to be untrusted by default. All traffic—lateral, outbound, or service-bound—must undergo explicit verification. This adheres to the Zero Trust doctrine of “*never trust, always verify*.”
- **IoT Device Limitations**: Devices are heterogeneous and may lack capabilities such as local cryptographic stacks, posture telemetry, or agent-based enrollment. The gateway assumes responsibility for trust mediation and policy enforcement on their behalf.
- **Gateway-Centric Enforcement**: All enforcement operations—posture validation, rule activation, segmentation—are consolidated at the consumer gateway. The system does not require SDN controllers, cloud-resident policy brokers, or enterprise PKI.
- **MUD-Enabled and Legacy Devices**: Devices that advertise signed MUD profiles (via DHCP Option 161 or LLDP) are treated with intent-aware enforcement. Devices lacking valid MUDs are placed into a default-deny segment with no implicit privileges.
- **Context-Aware Trust Evaluation**: Access control is not solely based on static declarations. The architecture continuously evaluates session posture, behavioral deviations, and profile validity during device operation, enabling context-aware enforcement.
- **Intermittent Connectivity Tolerance**: During brief controller outages, gateways cache validated policies and continue enforcement locally, ensuring resiliency and uninterrupted protection.
- **Secure Cryptographic Primitives**: Protocols such as WireGuard, SPA, and MUD signature verification are assumed secure. Physical device compromise is considered out-of-scope for this work.

These assumptions reflect the operating constraints of consumer deployments, where enforcement must function in real time, with minimal configuration, across diverse and potentially legacy IoT device classes.

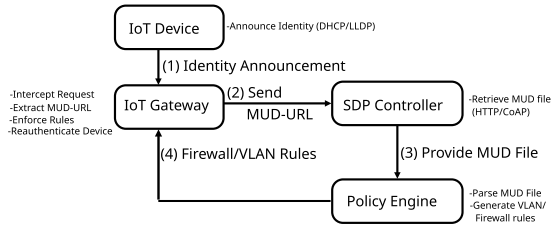


Fig. 3. MUD policy integration workflow: Device metadata is used to retrieve, validate, and enforce manufacturer-defined access control rules.

C. MUD Integration and Policy Generation

To enable automated and intent-driven access control, the architecture integrates the Manufacturer Usage Description (MUD) framework as defined by RFC 8520. MUD profiles are manufacturer-signed JSON documents that express expected device communication behavior, including permitted protocols, destinations, and ports. In the proposed architecture, these profiles establish the static intent foundation of the Zero Trust model.

Figure 3 illustrates the end-to-end MUD workflow from initial device detection to runtime enforcement.

Upon device onboarding, the gateway retrieves the associated MUD file using metadata obtained via DHCP Option 161 or LLDP advertisements. The profile is validated, parsed, and translated into per-device enforcement chains, including:

- iptables rules for directional flow control,
- VLAN tags for broadcast-domain isolation,
- WireGuard configurations for secure overlay tunneling.

Unlike policy frameworks that enforce access at coarse group granularity, the proposed system maintains per-device enforcement based on MAC/IP bindings. Even when multiple devices advertise the same MUD profile, enforcement is instantiated independently for each device instance, preventing privilege leakage due to misbehavior or compromise.

This per-device enforcement enables dynamic micro-segmentation: communication paths are activated only after successful posture validation and SPA approval, and policy violations trigger rapid revocation. As a result, MUD integration supports both static intent enforcement and dynamic trust adaptation—core principles of Zero Trust security.

1) MUD File Discovery and Retrieval: Upon initial network association, IoT devices may advertise metadata via DHCP Option 161/162 or LLDP as specified in RFC 8520, including a MUD-URL referencing a manufacturer-hosted policy profile. The IoT Gateway intercepts this metadata and forwards the MUD-URL to the SDP Controller, which functions as the Policy Decision Point (PDP).

The controller retrieves the MUD profile and verifies its digital signature using a preconfigured trust anchor (e.g., manufacturer CA) or DNS-based Authentication of Named Entities (DANE) when available. Signature verification ensures profile authenticity and integrity, preventing policy tampering during transit.

After validation, the MUD Policy Engine extracts communication intent, including:

- Permitted protocols and port ranges,
- Domain names or IP address ranges,

- Traffic directionality (from-device or to-device),
- Source and destination identity bindings.

These declarations are used as inputs to the dynamic rule-generation pipeline.

2) Policy Derivation Engine: The Policy Derivation Engine translates validated MUD declarations into actionable gateway-level enforcement logic. Each MUD entry yields a concrete rule set, including:

- Device-scoped iptables rules for transport-layer filtering,
- VLAN assignments for broadcast isolation,
- WireGuard peer entries for encrypted communication.

This automated derivation enables Zero Trust enforcement without manual rule configuration. For example, a smart thermostat authorized to access a cloud endpoint via TCP port 443 produces the following MUD entry:

Listing 1 Excerpt from MUD profile (RFC 8520 format).

```

1 {
2   "ietf-mud:mud": {
3     "mud-url": "https://vendor.com/mud/
4       thermostat1.json",
5     "from-device-policy": {
6       "access-lists": {
7         "acl-1": {
8           "aces": {
9             "ace-1": {
10              "matches": {
11                "ipv4": {
12                  "destination":
13                    "198.51.100.10",
14                  "protocol": 6,
15                  "destination-port": 443
16                }
17              },
18              "actions": {
19                "forwarding": "accept"
20              }
21            }
22          }
23        }
24      }
25 }
  
```

The resulting enforcement configuration at the gateway is: `iptables -A FORWARD -s 192.168.1.20 -d 198.51.100.10 -p tcpdport 443 -j ACCEPT`, followed by a default deny rule for all other outbound traffic.

All rules are placed in device-specific chains and remain active only while the device maintains a valid trust posture. Deviations result in rule revocation and enforcement teardown.

3) Scope of Traffic Inspection: Access control is enforced using packet metadata rather than application payload inspection. Evaluation criteria include:

- Source and destination IP/MAC addresses,
- Transport protocol and port number,
- Traffic directionality,
- Device identity bindings.

Deep packet inspection (DPI) is intentionally avoided due to gateway resource constraints and the widespread use of

encrypted protocols such as TLS/HTTPS, which make payload inspection impractical or privacy-invasive in consumer IoT environments.

Service discovery protocols such as mDNS and SSDP rely on multicast communication and are treated by the proposed architecture as ordinary network flows subject to the same Zero Trust enforcement logic. By default, discovery traffic is confined within device-specific VLANs and is denied across trust boundaries, preventing multicast-based discovery from enabling unintended lateral visibility or communication. As with unicast traffic, any discovery-related flow must be explicitly permitted by both MUD-derived constraints and dynamic SDP authorization; otherwise, it is dropped by the gateway.

4) *Micro-Segmentation Policy Application:* Dynamic micro-segmentation is enforced through per-device isolation mechanisms. Each device is assigned a logical segment based on its identity and MUD-derived policy scope, and inter-segment communication is permitted only when explicitly authorized.

While VLAN-based micro-segmentation is used in the proposed architecture due to its wide availability and low overhead on consumer gateways, the design does not preclude more advanced policy abstraction mechanisms. Software-Defined Networking (SDN) or EVPN-based segmentation can offer finer-grained and centrally managed policy abstractions in enterprise environments; however, such approaches require additional control-plane infrastructure and persistent controller connectivity. These assumptions do not hold in unmanaged residential IoT settings, where simplicity, local enforcement, and fail-safe behavior are primary design constraints. We therefore adopt VLANs as a pragmatic enforcement substrate while maintaining a policy model that can be extended to SDN or EVPN backends in future deployments.

Segmentation enforcement includes:

- Device-specific `iptables` chains,
- MAC/IP bindings to prevent spoofing,
- VLAN-based broadcast isolation,
- Conditional WireGuard tunnels based on trust posture.

For example, a smart bulb authorized only to access a manufacturer cloud API is denied communication with peer devices or unrelated internet hosts, even if other devices share its VLAN or model. This enforces least-privilege routing and prevents lateral movement.

5) *Policy Refresh and Caching:* Validated MUD profiles are cached locally and refreshed upon firmware updates, device resets, manufacturer-specified expirations, or periodic revalidation during DHCP lease renewals and SPA posture checks. Profile retrieval is protected using TLS/DTLS as specified in RFC 9761, ensuring secure and adaptive enforcement without administrator intervention.

6) *Failure Handling: Malformed, Unreachable, or Expired MUD URLs:* To preserve security under policy resolution failures, the system defaults to fail-secure behavior.

Failure scenarios and actions:

- **Malformed MUD URL:** Device is placed in a quarantine VLAN with minimal connectivity (e.g., DHCP only).
- **Retrieval failure:** Cached profiles are reused temporarily when available; otherwise, enforcement defaults to deny.

- **Signature failure or expired profile:** The profile is discarded and no access rules are instantiated.
- **Tunnel establishment failure:** The device is immediately isolated to prevent fallback access.

All failure events are logged, and no implicit privileges are granted.

7) *Mitigating Over-Permissive and Stale MUD Profiles:*

The proposed architecture does not assume that manufacturer-provided MUD profiles are inherently correct, minimal, or up to date. Instead, MUD declarations are treated as *upper-bound intent constraints*, not unconditional trust grants.

Over-permissive MUD profiles are bounded by the dynamic enforcement layer. Even if a MUD profile permits a broad set of destinations or services, all communication remains subject to session-level authorization, posture validation, and contextual SDP policies. As a result, static manufacturer intent can never override runtime trust evaluation or elevate privileges beyond what is currently validated.

To address stale or outdated MUD profiles, the system incorporates periodic revalidation triggered by DHCP lease renewal, device reattachment, or manufacturer-specified expiry metadata. Cached MUD profiles are refreshed opportunistically, and enforcement defaults to fail-secure behavior if a profile cannot be revalidated. Devices operating under outdated or unverifiable profiles are confined to a restricted segment until validation succeeds.

In addition, the gateway supports local policy overrides that allow administrators to further constrain MUD-derived permissions (e.g., limiting destination domains or protocols) without modifying the original profile. These overrides operate as safety guards and do not require manual rule scripting, ensuring that misaligned or overly broad manufacturer intent cannot expand the effective attack surface.

By combining static intent bounding with dynamic authorization and local override capability, the architecture mitigates risks arising from over-permissive or stale MUD profiles while preserving the benefits of automated, standards-compliant policy generation.

8) *Audit Logging and Policy Export for Compliance:*

The MUD Controller logs all enforcement actions—including VLAN assignments, `iptables` rule insertions, WireGuard peer setups, and policy revocations—with precise timestamps in structured JSON format to support audit traceability.

Logs can be exported in standardized formats, including JSON-LD for semantically rich policy representation and STIX 2.1 for SIEM and threat-correlation pipelines. Real-time forwarding to external monitoring systems (e.g., ELK/EFK, Splunk, QRadar) enables cross-gateway correlation, violation detection, and post-incident analysis.

Figure 4 summarizes the full MUD-based policy derivation and enforcement process.

D. SDP Controller and Gateway Enforcement

The Software-Defined Perimeter (SDP) Controller acts as the central Policy Decision Point (PDP), making real-time access decisions based on SPA posture, MUD intent, and contextual evaluation. The IoT Gateway serves as the Policy

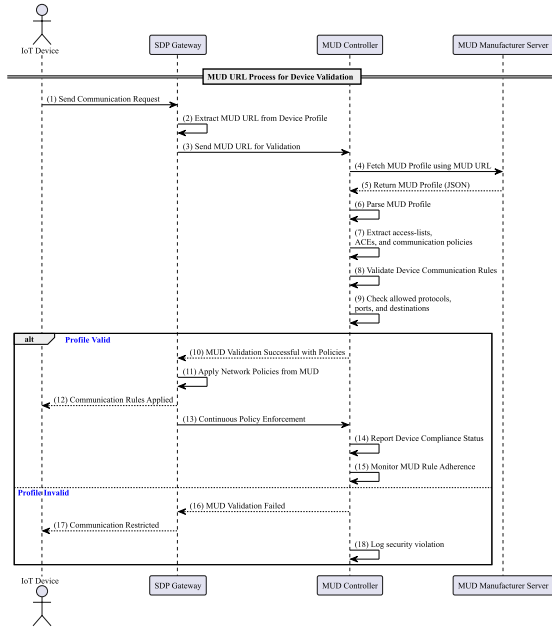


Fig. 4. MUD policy derivation workflow: Runtime enforcement based on manufacturer-defined access controls.

Enforcement Point (PEP), executing these decisions through VLAN assignment, firewall rules, and WireGuard tunnel configuration.

Figure 5 illustrates the step-by-step flow from device onboarding to secure communication:

1) *Context-Aware Policy Resolution*: At runtime, enforcement adheres to a conflict resolution hierarchy where dynamic trust context overrides static declarations:

- 1) **Admin Policies (Highest Precedence)**: Explicit per-device overrides or global rules.
- 2) **Contextual SDP Policies**: SPA validation, posture scoring, identity attributes.
- 3) **MUD Policies**: Manufacturer-declared communication constraints.

If a MUD profile permits a flow but the current SPA posture is non-compliant, the system blocks the flow and transitions the device to a restricted state. All decisions are logged for auditing and compliance.

This layered model ensures static MUD declarations never override dynamic trust evaluation, in full compliance with NIST SP 800-207 Zero Trust principles.

E. Formal Policy Enforcement Logic

1) *Flow Evaluation Semantics*: Each flow is evaluated against both static policy and dynamic session context. Let:

- d : A specific IoT device
- P_d : Set of permitted flows from MUD
- f : Observed traffic flow $\langle src, dst, proto, port \rangle$
- $\pi(f)$: Policy decision result ($\pi(f) = \text{SDP}_{\text{gateway}}(f)$)

Then:

$$\pi(f) = \begin{cases} \text{ALLOW}, & f \in P_d \wedge \text{session_valid}(f) \\ \text{DROP}, & f \notin P_d \\ \text{REVOKE}, & \text{if posture_compliant}(d) = \text{false} \end{cases}$$

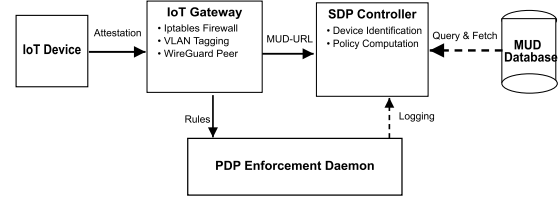


Fig. 5. SDP enforcement flow: The controller validates device identity, computes access rules from the MUD profile, and issues VLAN, iptables, and WireGuard configurations to the IoT Gateway.

This ensures that authorization is continuously re-evaluated, not statically assumed.

2) *Rule Generation Pipeline*: For all permitted flows $f \in P_d$, the system generates a corresponding set of enforcement artifacts E :

$$\begin{aligned} E &= \phi(P_d) \\ &= \bigcup_{f_i \in P_d} \text{iptables}(f_i) \cup \text{VLAN}(d) \cup \text{wg_peer}(d) \end{aligned}$$

Where $\mathcal{V}$

- $\text{iptables}(f_i)$ — flow-level filtering rules,
- $\text{VLAN}(d)$ — device isolation tag,
- $\text{wg_peer}(d)$ — WireGuard tunnel binding for encryption.

These artifacts are applied atomically at the gateway by a dedicated enforcement daemon. If an unauthorized or posture-invalid flow $f' \notin P_d$ is detected $\mathcal{V}$

$$\pi(f') = \text{DROP} \quad \text{and} \quad d \rightarrow \text{quarantine}$$

This transition revokes WireGuard keys, removes policy bindings, and places the device into a deny-all VLAN segment. All enforcement actions are synchronously logged and exported.

3) *Time-Bounded Enforcement*: Let T_{detect} be the latency for detecting a posture or policy violation, and T_{apply} the time to enforce a rule update. We define the end-to-end containment latency as $\mathcal{V}$

$$T_{\text{react}} = T_{\text{detect}} + T_{\text{apply}} \leq 300 \text{ ms}$$

This enforcement bound is validated in Section IV and ensures that misbehaving flows are blocked before they can complete unauthorized data exchanges.

4) *Compliance Alignment*: This enforcement model satisfies the key tenets of Zero Trust by ensuring that:

- Trust is evaluated per session and never retained implicitly,
- Flows are scoped strictly to manufacturer-declared behavior,
- All enforcement actions are auditable and traceable to declarative policy sources.

5) *Device Identification and Attestation*: Upon initial network join, the gateway may extract device metadata using DHCP Option 161/162 or LLDP announcements, including MUD-URLs. This metadata is forwarded to the SDP Controller for retrieval and integrity verification.

To further strengthen trust attribution, the architecture supports:

- Certificate-based onboarding using device identity certificates,
- TPM-backed attestation for integrity proofs,
- MAC/OUI fingerprinting to map vendors and enforce manufacturer scoping.

Unrecognized or unregistered devices may be subject to administrator-mediated approval, including optional multi-factor authentication (MFA), prior to policy activation.

6) *Dynamic Policy Binding*: Once identity and intent are verified, the SDP Controller synthesizes a dynamic enforcement state that includes:

- Device-specific VLAN assignment,
- Scoped `iptables` filtering for allowed destinations and services,
- Per-session WireGuard key and peer binding for tunnel enforcement.

These instructions are relayed securely to the IoT Gateway, where they are applied using Linux bridging, routing, and rule insertion interfaces. Enforcement is isolated per device instance.

7) *Real-Time Rule Enforcement*: Enforcement occurs continuously at the gateway using a dedicated Enforcement Daemon, which:

- Applies `iptables` or `nftables` entries for traffic control,
- Maintains WireGuard tunnels for flow encryption,
- Uses `ip link` and `vconfig` for VLAN tagging.

Each device is treated as a unique trust boundary. Lateral traffic is blocked by default unless dual policy permissions (MUD + SDP) explicitly allow it [24].

8) *Atomic Policy Updates and Rollback Safety*: All enforcement updates follow an atomic commit model to prevent partial rule application. The process:

- Stages all updates in memory,
- Applies them in a single transaction,
- Performs validation and post-application confirmation.

If a failure occurs (e.g., memory exhaustion, key conflict), a rollback routine restores prior state, moves the device to a restricted VLAN, and logs the incident for forensic auditing.

When MUD profiles are updated during active communication sessions, the same atomic enforcement model applies. If an updated MUD profile does not invalidate the permissions of an ongoing session, the session continues uninterrupted. If required permissions are revoked, the change is enforced at the next policy evaluation boundary by gracefully terminating the affected session through WireGuard key revocation and rule removal. This ensures consistent enforcement without breaking compliant flows or leaving stale permissions active.

9) *Runtime Adaptability and Policy Updates*: Figure 6 illustrates the system's behavior when receiving manufacturer initiated MUD updates or detecting behavioral deviation.

The policy update flow is as follows:

- MUD Controller detects or is notified of a profile update.
- The new profile is retrieved and verified.
- Updated enforcement rules are derived and sent to the gateway.

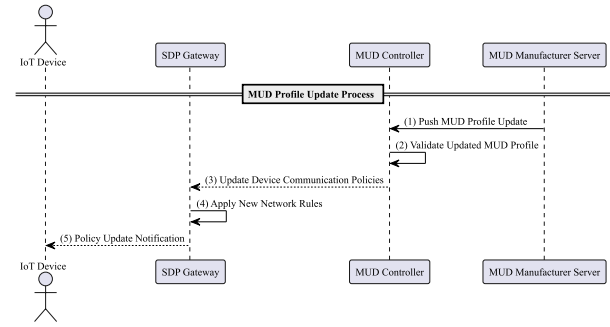


Fig. 6. Runtime adaptability workflow in response to updated MUD profiles or observed posture changes.

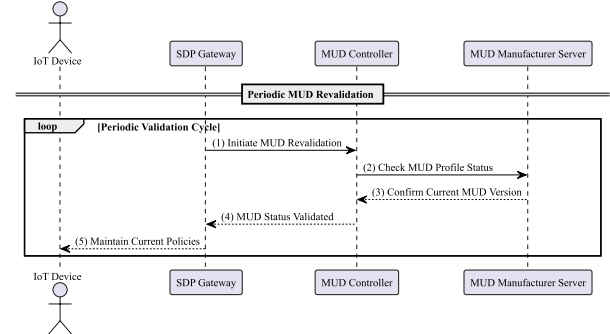


Fig. 7. Continuous trust enforcement cycle with periodic MUD validation and behavioral compliance monitoring.

- The gateway applies rule updates without disrupting active valid sessions.

10) *Policy Lifecycle and Expiry*: Policy validity is tied to temporal and session-related triggers:

- DHCP lease cycles,
- Active session timers or posture deviation,
- Manufacturer policy versioning or expiry flags.

The SDP Controller logs all lifecycle state transitions for audit traceability and compliance enforcement under secure-by-default mandates.

11) *Session Ephemerality and Keepalive Enforcement*: All access sessions are ephemeral. Session validity is periodically revalidated using SPA keepalives containing posture and identity metadata. If the SDP Controller fails to receive these within the allowed timeout or posture changes unfavorably:

- WireGuard sessions are revoked,
- Associated enforcement rules are purged,
- Devices are quarantined until revalidation.

This continuous validation loop ensures trust is scoped to the session, not the device identity.

12) *Device Onboarding Workflow*: Onboarding is automated and spans several trust-building phases, as illustrated in Figure 7.

Phases:

- 1) **Discovery**: Detected via DHCP or Layer 2 broadcast.
- 2) **Identity Attribution**: Mapped using MAC/OUI, DHCP fingerprinting, DNS, or behavioral metadata.
- 3) **MUD Retrieval**: MUD-URL is resolved; profile is verified.
- 4) **Policy Derivation**: Device permissions are computed from profile content.

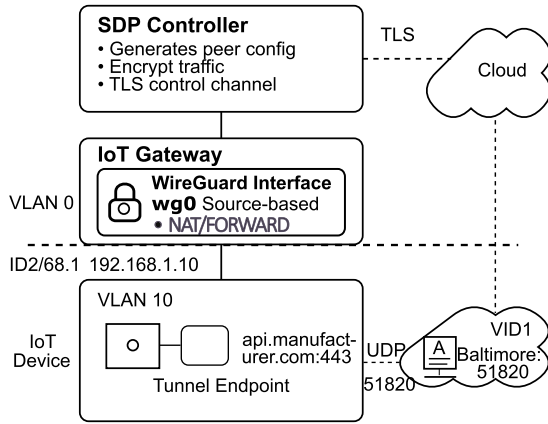


Fig. 8. Secure routing and flow enforcement with WireGuard. Permitted flows are encrypted and routed through interface-specific tunnels; unauthorized flows are dropped or quarantined.

5) **Conditional Activation:** On first communication, an SPA is sent and access is granted only if trust context passes.

6) **Continuous Evaluation:** Ongoing posture checks and behavioral monitoring enforce dynamic revalidation.

Devices without valid MUDs are sandboxed in a restrictive VLAN with only DNS/DHCP allowed until admin review or recovery.

F. WireGuard Tunneling and Flow Isolation

To ensure secure transmission, authorized flows are tunneled using WireGuard [25], a performant, minimalist VPN protocol suitable for embedded use cases. Figure 8 illustrates the routing and enforcement model.

1) *Peer Establishment and Key Exchange:* Per-session WireGuard configurations are generated by the SDP Controller:

- Unique key pairs per session,
- Flow-specific IP/port allowlists,
- Endpoint metadata and tunnel identifiers.

Only posture-compliant devices receive key material. All key events are ephemeral and logged.

2) *Interface Binding and Routing Rules:* WireGuard peers are attached to interfaces (e.g., wg0, wg1) linked to VLAN and device sessions. Routing logic is enforced through:

- Custom `ip rule` tables for source routing,
- Flow-specific `iptables` chains with tunnel binding.

This forms device-specific micro-perimeters.

3) *Policy-Constrained Encryption:* Tunnels are constrained strictly to flows permitted by both MUD and dynamic SDP context. For example, if a device is allowed only HTTPS traffic to `api.vendor.com`, only that traffic is routed through the tunnel; all else is dropped.

4) *Scalability and Performance:* Testing on a Raspberry Pi 4B with 4 GB RAM showed:

- >90 Mbps encrypted throughput,
- <20% CPU usage under active tunneling,
- <200 ms tunnel instantiation time.

These results validate that strong flow isolation and encryption can be enforced on consumer-grade hardware.

5) *Key Lifecycle and Revocation:* Key material is ephemeral and regenerated:

- Per session,
- Upon posture invalidation,
- After timeouts or inactivity (default: 10 min).

In the event of compromise:

- Keys are revoked,
- Session is torn down,
- Device is moved to quarantine.

Optional proactive rekeying ensures forward secrecy.

6) *NAT Traversal and Tunnel Liveness:* Persistent NAT traversal is supported via:

- PersistentKeepalive at 25s intervals,
- Adaptive jitter to reduce fingerprinting,
- Optional STUN-like fallback mechanisms for Carrier-Grade NATs.

These ensure tunnel availability even in constrained or asymmetric home networks.

G. End-to-End Workflow

This section outlines the full operational lifecycle of an IoT device under the proposed Zero Trust architecture—from discovery and policy retrieval to conditional activation and dynamic revocation. The process is designed to be fully automated, failure-resilient, and administrator-light, supporting deployment in unmanaged consumer environments.

1) *Step 1: Device Discovery and Metadata Capture:* Upon network association, the IoT device emits DHCP (Option 161/162) or LLDP frames containing a MUD-URL and identity metadata (MAC, IP, OUI). The IoT Gateway intercepts this metadata to initiate policy lookup and classification.

2) *Step 2: MUD Retrieval and Validation:* The SDP Controller fetches the referenced MUD file via HTTP or CoAP. The profile is validated for schema correctness and authenticity using its digital signature. If retrieval fails or signature verification is unsuccessful:

- The device is quarantined in a default-deny VLAN.
- Only essential traffic (DHCP, DNS) is allowed.
- Cached profiles may be used temporarily, but onboarding halts until validation is successful or administrative override is applied.

This strict fail-closed approach ensures incomplete onboarding does not result in implicit trust.

3) *Step 3: Policy Derivation and Enforcement Logic:* Upon validation, the MUD Controller parses the file to extract enforceable communication intent:

- Allowed destinations (IP/domain), ports, and directions,
- Required segmentation scope (e.g., VLAN group),
- Encryption constraints (e.g., tunneling necessity).

The SDP Controller compiles this into device-specific enforcement artifacts.

4) *Step 4: Policy Deployment:* The Controller securely transmits the enforcement plan to the IoT Gateway, which applies:

- VLAN tagging via Linux bridging tools,
- Scoped `iptables` rules for protocol and flow restriction,

- WireGuard tunnel configuration for authorized encrypted flows.

All enforcement steps are logged with timestamps for compliance auditing.

5) *Step 5: Runtime Monitoring and Policy Refresh*: Active sessions are monitored for deviations in behavior or posture. Triggers for policy revalidation include:

- DHCP lease renewal,
- Manufacturer profile updates,
- New outbound or lateral connection attempts.

Any deviation prompts real-time reevaluation and potential rule modification.

6) *Step 6: Policy Revocation and Quarantine*: Upon detecting a policy violation or posture failure, the Gateway immediately:

- Revokes the WireGuard peer key and session,
- Removes corresponding iptables and VLAN assignments,
- Reassigns the device to a quarantine VLAN under a default-deny regime.

Notifications are emitted to the audit log to support regulatory compliance and post-incident analysis.

7) *Two-Stage Evaluation for Inter-Device Traffic*: To prevent implicit trust among co-resident IoT devices, lateral communication undergoes a two-stage Zero Trust evaluation.

- *Flow Interception*: Device 1 initiates a flow to Device 2; the Gateway intercepts the request.
- *Static Validation (Stage 1)*: The MUD Controller checks whether the flow is within Device 1's declared behavior. If not, it is dropped.
- *Dynamic Trust Evaluation (Stage 2)*: If allowed by MUD, the SDP Controller assesses current posture, user context, and session metadata before authorizing a temporary WireGuard tunnel and firewall rules.
- *Scoped Activation*: Validated sessions are permitted with ephemeral credentials and strict timeouts.

This dual-stage process ensures that static device claims do not bypass runtime posture enforcement, achieving granular containment.

8) *Control Plane Resilience and DoS Defense*: SPA and policy infrastructure are hardened against overload and denial attacks using:

- **Rate-limited ingress**: eBPF/iptables token-bucket filters per source IP,
- **Stateless filtering**: SPA packets validated with timestamps, nonces, and MACs before state allocation,
- **Queue capping**: Maximum bounds on pre-auth queues with source-based backoff,
- **Greylisting**: Misbehaving sources are deprioritized without blacklisting,
- **Plane isolation**: Separate interfaces and CPU quotas for data and control traffic,
- **Key management hardening**: Keepalive timing is randomized to reduce attack predictability,
- **Failover model**: Active-passive SDP controllers ensure continuity; devices default to least-privilege on failure,

- **Safety invariants**: Unprovisioned devices remain isolated; no new privileges are issued without policy and SPA revalidation.

9) *Hardware-Rooted Attestation and Trust Anchors*: For devices supporting TPM 2.0 or secure elements, hardware-rooted identity and integrity can be incorporated using IETF RATS and IEEE 802.1AR standards.

TPM-enabled Attestation:

- Devices sign PCR values (firmware, OS) using AIK,
- SDP Controller validates the attestation report and freshness nonce over TLS,
- Mismatches or untrusted measurements result in quarantine.

Certificate-Based Bootstrapping:

- Devices present X.509 credentials during mTLS handshake,
- The Controller validates issuer trust, revocation status, and binds identity to enforcement policy.

These mechanisms are integrated into the continuous posture validation loop. Any lapse in attestation freshness or trust state triggers immediate access revocation and re-isolation. This ensures persistent compliance with the Zero Trust mandate of continuous verification.

IV. EXPERIMENTAL EVALUATION

The evaluation is structured around five dimensions: performance on constrained hardware, dynamic policy adaptation capability, effectiveness of Zero Trust enforcement under lateral and outbound threats, comparative benchmarking against established Zero Trust frameworks, and administrative overhead reduction through automation. To emulate heterogeneous IoT environments, our testbed includes microcontroller-class devices (ESP32), Linux-class IoT nodes (Raspberry Pi Zero W), and replay-based emulations of camera and thermostat traffic, representing high-frequency UDP telemetry, HTTPS polling, and long-lived TCP sessions. This setup reflects the protocol and communication diversity typical of consumer IoT deployments. Although MUD profiles are declarative and static by design (RFC 8520), the proposed architecture integrates DHCP-triggered revalidation, runtime SPA-driven posture checks, and manufacturer-initiated policy updates to support adaptive enforcement. This bridges static intent declarations with dynamic Zero Trust validation at runtime.

A. Performance and Scalability on Constrained Gateways

Most Zero Trust frameworks assume enterprise-grade hardware with ample compute resources and centralized administrative control. In contrast, we evaluate whether full-stack enforcement—including SPA-triggered tunnels, MUD-aware segmentation, and policy updates—can be sustained on low-cost, resource-constrained gateways typical of consumer IoT environments. This comparison is critical to demonstrate the feasibility of deploying Zero Trust enforcement outside datacenter settings, where enterprise frameworks (e.g., BeyondCorp, OpenZiti) or SDN-based architectures often rely on high-performance hardware and persistent control planes.

TABLE I

PERFORMANCE METRICS OF THE PROPOSED MUD-SDP ARCHITECTURE ON A RASPBERRY PI GATEWAY, DEMONSTRATING LOW-LATENCY SESSION ESTABLISHMENT AND RESOURCE-EFFICIENT ZERO TRUST ENFORCEMENT UNDER TYPICAL IOT WORKLOADS

Metric	Value	Interpretation
Tunnel Setup Time	185 ms	Real-time session provisioning
WireGuard Throughput	92 Mbps	Efficient encrypted transport
CPU Utilization	18% avg	Low processing overhead
Round-Trip Latency	6.4 ms	Supports time-sensitive flows
Policy Update Time	850 ms	Fast MUD policy re-evaluation
Rules Insertion Time	95 ms	Lightweight gateway enforcement
Memory Footprint	42 MB	Minimal memory consumption

To assess feasibility in this setting, we deployed the MUD-SDP enforcement stack on a Raspberry Pi 4B gateway (4 GB RAM), representative of smart home routers and IoT hubs.

1) *Testbed Configuration*: The gateway enforces MUD-derived policies using `iptables`, VLAN isolation via `bridge-utils`, and WireGuard tunneling. The SDP Controller, hosted on an Intel Core i5-10500 system, runs the MUD engine, SPA validator, rule generator, and REST APIs. Device heterogeneity is represented using six Raspberry Pi Zero W nodes and two ESP32 microcontrollers, with scaled workloads, emulating classes such as IP cameras, smart plugs, lights, and thermostats. Behavioral realism is achieved through replayed packet traces from commercial IoT products. The network fabric consists of IEEE 802.11n Wi-Fi and 1 Gbps Ethernet, with devices advertising MUD URLs via DHCP Option 161 or LLDP in compliance with RFC 8520.

2) *Gateway Resource Metrics*: We evaluated gateway resource usage and responsiveness under peak simulated IoT activity. Results are summarized in Table I.

The Raspberry Pi gateway sustained encrypted forwarding at 92 Mbps with average CPU utilization below 20%, confirming that full Zero Trust enforcement—including tunneling, micro-segmentation, and policy recomputation—remains viable on consumer-grade hardware. Compared to enterprise-focused frameworks such as BeyondCorp or OpenZiti, which assume x86-class edge nodes and administrator-managed provisioning, the proposed system achieves comparable enforcement performance on a significantly more constrained platform. Prior academic systems based on SDN or NFV frequently report latency and CPU overheads unsuitable for real-time consumer deployments. These results demonstrate that secure session tunneling, per-device policy enforcement, and runtime updates can be achieved within tight resource budgets.

B. Dynamic Adaptation and Policy Update Latency

A key challenge in consumer Zero Trust enforcement is limited support for dynamic policy realignment. Frameworks based solely on MUD or static ACLs typically require manual intervention to reflect behavioral changes or updated manufacturer intent. In contrast, the proposed architecture supports runtime adaptation through profile refresh, SPA posture reassessment, and lease-driven enforcement resets.

To evaluate adaptive behavior, we measured *runtime adaptation delay*, defined as the interval from MUD profile change detection to complete enforcement update, encompassing:

- Retrieval of updated MUD profiles,
- Parsing and validation at the controller,
- Rule generation (`iptables`, VLAN, WireGuard),
- Deployment at the gateway.

Across 30 update iterations, adaptation delay averaged 872 ms (± 35 ms), confirming sub-second end-to-end realignment.

Policy updates in the proposed system are primarily event-driven, triggered by device attachment, MUD profile resolution, session authorization, or policy revocation events. Under normal operation, rule updates are infrequent and limited to these discrete events, resulting in low steady-state churn.

To assess bursty join scenarios, we consider the cumulative overhead of sequential policy instantiation for multiple devices joining within a short time window. Since each device is handled independently and policy updates are applied atomically, bursty joins result in a linear increase in control-plane activity without introducing contention or inconsistent enforcement states. Our measured per-device rule installation and enforcement latency therefore represent the worst-case cost for each join event.

In practice, gateway CPU utilization remains bounded, and enforcement correctness is preserved even during rapid device onboarding. No global recomputation or controller synchronization is required, making the approach suitable for residential environments with occasional bursty device joins.

To prevent stale policies, active sessions are purged upon DHCP lease expiry, removing associated `iptables` rules, WireGuard bindings, and cached posture state. Future extensions will integrate behavior-driven expiry triggers and signed MUD revocations to support finer-grained validity windows.

These results contrast with prior ZTA systems where policy propagation times often exceed one second due to cloud orchestration or manual rule updates. By achieving sub-second adaptation using local enforcement on a Raspberry Pi 4B, the proposed architecture demonstrates that continuous Zero Trust validation is feasible in unmanaged consumer environments.

C. Zero Trust Enforcement Effectiveness and Backend Impact

Zero Trust enforcement requires not only correct policy specification but also efficient execution across enforcement backends. Many frameworks rely on cloud-based controllers or SDN switches for rule enforcement. In contrast, our design evaluates the performance of standard Linux firewalls on low-cost hardware. We replicated the MUD-SDP enforcement workload using both `iptables` and `nftables` on the same Raspberry Pi 4B gateway, enforcing 100 per-device rules across 20 active IoT endpoints. Results are shown in Table II.

The results show that `nftables` offers moderate efficiency improvements, while `iptables`—the reference backend in MUD specifications—remains sufficient for dynamic enforcement at scale. By demonstrating comparable performance across both backends, we confirm that Zero Trust enforcement

TABLE II

COMPARISON OF IPTABLES AND NFTABLES ENFORCEMENT BACKENDS ON A RASPBERRY PI GATEWAY, HIGHLIGHTING REDUCED RULE INSERTION LATENCY AND LOWER CPU OVERHEAD WITH NFTABLES

Metric	iptables	nftables	Improvement
Rule Insertion Time (avg)	95 ms	68 ms	28.4% ↓
Enforcement Latency (per flow)	285 ms	252 ms	11.6% ↓
CPU Usage @ 20 Devices	32%	26%	18.8% ↓
Memory Footprint	42 MB	34 MB	19.0% ↓
Peak Rule Evaluation Rate	12K/s	16K/s	33.3% ↑

can be implemented using standard Linux tooling without proprietary agents or specialized hardware.

1) *Bandwidth Overhead From SPA Keepalives*: The architecture uses lightweight SPA keepalives for continuous posture validation. Each keepalive packet is approximately 512 bytes and transmitted every 25 seconds, resulting in 20.5 bytes/sec per device. For 50 devices, aggregate overhead remains below 1.03 KB/sec.

Compared to agent-based frameworks requiring persistent connections or active client daemons, this approach maintains trust state using minimal, stateless probes. This enables continuous Zero Trust enforcement with negligible bandwidth and energy overhead in typical smart home deployments.

D. Security Enforcement Under Lateral and Outbound Threats

Enforcement fidelity under adversarial conditions is central to validating any Zero Trust implementation. Prior systems often demonstrate IP-layer policy enforcement but struggle with intra-LAN threats, dynamic lateral scans, and Layer 2 attacks—particularly in unmanaged environments such as smart homes. We evaluate how the proposed MUD–SDP architecture responds to these challenges and compare it with declarative-only MUD enforcement and conventional ACL-based filtering.

To validate Zero Trust guarantees, we emulated unauthorized communication attempts across multiple threat vectors, including lateral device-to-device probes, outbound exfiltration attempts, and Layer 2 spoofing.

1) *Lateral Movement Prevention*: Attempts by a smart light to initiate unsolicited communication with a camera device were consistently blocked at the gateway, confirming effective micro-segmentation.

2) *Unauthorized Outbound Communication*: Outbound traffic targeting unauthorized IPs and ports (e.g., social media or public DNS services) was denied and logged through the MUD–SDP enforcement chain.

The observed 1.6% miss rate is attributed to transient race conditions during DHCP lease renewal and short-lived multicast bursts prior to full rule instantiation. These events do not result in sustained unauthorized access.

3) *Reaction Time to Violations*: Average time from policy violation to active session revocation and rule purge remained below 300 ms, aligning with the enforcement bound.

TABLE III

EFFECTIVENESS OF ZERO TRUST ENFORCEMENT UNDER LATERAL MOVEMENT, OUTBOUND POLICY VIOLATION, AND SPOOFING ATTACK SCENARIOS, DEMONSTRATING HIGH BLOCKING ACCURACY AND SUB-300 MS REACTION TIME

Metric	Value	Interpretation
Lateral Movement Block Rate	100%	No peer-to-peer flows unless explicitly permitted
Policy Violation Detection	98.4%	Unauthorized outbound flows dropped
Enforcement Reaction Time	285 ms	Rapid response to violations
Flow Coverage (Blocked/Total)	96.7%	High coverage without tuning

TABLE IV

COMPARISON OF ENFORCEMENT LATENCY, ONBOARDING TIME, AND RESOURCE OVERHEAD BETWEEN THE PROPOSED MUD–SDP ARCHITECTURE AND REPRESENTATIVE ENTERPRISE ZERO TRUST FRAMEWORKS UNDER THEIR RESPECTIVE DEPLOYMENT ASSUMPTIONS

Metric	Our System	Google BeyondCorp	OpenZiti
Enforcement Latency (per flow)	285 ms	330–400 ms	312 ms
Device Onboarding Time	1.8 s	3.2 s	2.5 s
Policy Update Propagation	872 ms	~1 s	950 ms
Client Resource Footprint	7.3 MB	N/A (browser agent)	11.2 MB
Gateway CPU (20 devices)	32%	—	45%

4) *Layer 2 Enforcement and Identity Binding*: VLAN segmentation isolated broadcast domains, while MAC-to-IP bindings were enforced using eBPF and iptables. SPA keepalive authentication ensured that session continuity remained tied to policy-compliant posture.

To evaluate resilience against Layer 2 evasion, we simulated ARP spoofing and MAC flooding using `arpspoof`. VLAN isolation and eBPF-assisted filtering prevented forged ARP replies and dropped excessive broadcast traffic before it could disrupt legitimate sessions. No cross-segment leakage was observed, confirming that the enforcement mechanisms extend Zero Trust guarantees beyond the IP layer.

5) *Aggregate Coverage*: Under stress conditions, 96.7% of attempted unauthorized flows were blocked automatically without administrator intervention. Table III summarizes enforcement effectiveness.

Compared to MUD-only enforcement and SDN-based policy systems that rely on controller synchronization or lack Layer 2 context, the proposed architecture demonstrates effective decentralized enforcement. Gateway-based segmentation with eBPF augmentation provides real-time protection against lateral movement and spoofing without centralized orchestration, making it well suited for secure residential environments.

E. Comparative Benchmarking With Enterprise ZTA Frameworks

To contextualize performance, we benchmarked the proposed MUD–SDP architecture against enterprise-grade Zero Trust frameworks—Google BeyondCorp and OpenZiti—which assume powerful gateway infrastructure,

TABLE V

REDUCTION IN ADMINISTRATIVE CONFIGURATION EFFORT ACHIEVED THROUGH AUTOMATED MUD-SDP POLICY DERIVATION COMPARED TO MANUAL VLAN AND FIREWALL CONFIGURATION

Metric	Value	Interpretation
Total Config Time (Manual)	42 min	Static ACL configuration for 8 devices
Total Config Time (MUD-SDP)	9 min	MUD-driven automated policy activation
Time Saved per Device	4.1 min	77–78% faster provisioning
Auto-Generated Firewall Rules	84 rules	No manual editing required
VLAN Tags Assigned Automatically	8 VLANs	Profile-to-policy mapping successful

persistent client agents, and managed administrative control. In contrast, our system targets autonomous operation on constrained smart home gateways with heterogeneous IoT devices and no client-side software. Table IV summarizes the comparison. Figures for BeyondCorp and OpenZiti were obtained from public technical reports and prior benchmark studies [26], [27]. Despite operating on Raspberry Pi 4B hardware, the proposed system delivers enforcement latency and propagation times comparable to enterprise frameworks that rely on x86-class gateways. The absence of persistent client agents significantly reduces client-side overhead, confirming that adaptive Zero Trust enforcement can be achieved in unmanaged consumer environments without sacrificing responsiveness.

a) *Comparison with Related Zero Trust Frameworks:* To contextualize the effectiveness of the proposed MUD-SDP architecture, we compare it qualitatively with representative Zero Trust enforcement approaches such as Mudscan and blockchain-based ZTNA frameworks. Mudscan primarily strengthens device onboarding through double authentication and MUD validation, but its enforcement model remains largely static after admission and lacks continuous session-level revocation. In contrast, the proposed system enforces dynamic, per-session authorization, allowing active flows to be revoked within sub-300 ms upon posture deviation or policy violation. Blockchain-based ZTNA approaches provide strong auditability and immutable policy logging; however, they introduce additional control-plane latency, synchronization overhead, and infrastructure dependencies that are unsuitable for resource-constrained residential gateways. Our gateway-centric design avoids ledger interaction and cloud dependence, enabling lightweight runtime enforcement using standard Linux primitives (iptables, VLANs, and WireGuard). Overall, this comparison highlights that the proposed architecture achieves higher enforcement fidelity through continuous authorization and rapid reaction, while maintaining lower computational and operational overhead compared to existing MUD-only or blockchain-backed Zero Trust frameworks.

F. Administrative Overhead Reduction and Operator Efficiency

A key differentiator of our architecture is the automation of enforcement policy deployment. Whereas enterprise

TABLE VI

COMPARISON OF POLICY VIOLATION REACTION TIMES ACROSS REPRESENTATIVE ZERO TRUST ARCHITECTURES, HIGHLIGHTING THE FASTER CONTAINMENT ACHIEVED BY THE PROPOSED GATEWAY-CENTRIC ENFORCEMENT MODEL

Framework	Enforcement Mechanism	Reported Reaction Time
Proposed MUD-SDP (this work)	WireGuard + iptables micro-segmentation	285 ms (Raspberry Pi 4B)
Morris & Kornegay (2025) [7]	Flow-table modification in SDN switches	~500 ms
Poirrier et al. (2025) [6]	SDP trust boundary enforcement	400–700 ms
Kaur et al. (2025) [16]	Blockchain-based Zero Trust for fog computing	Seconds (high overhead)
Majid et al. (2024) [17]	NFV + cognitive SDN routing	~600 ms
Datta et al. (2025, Mudscan) [15]	Double authentication with MUD profiles	>600 ms

Zero Trust frameworks assume manual rule definition or pre-installed agents, our approach automatically extracts MUD-based intent and applies enforcement without administrator input. We quantified the configuration effort required for 8 representative IoT devices under two scenarios: (1) manual VLAN tagging and ACL rule definition, and (2) our MUD-SDP enforcement pipeline. The difference is captured in Table V. Unlike systems such as BeyondCorp, which require central control and active directory integration, our system deploys policy from device declaration (via MUD) to enforcement in under 10 minutes for a full network. This reduction in provisioning effort enables secure-by-default operation, even in homes without IT expertise—a key enabler for Zero Trust democratization.

G. Cross-Framework Enforcement Reaction Time Analysis

To further evaluate enforcement responsiveness, we compared our system’s reaction time against a range of Zero Trust frameworks and academic proposals. Reaction time is defined as the interval between detection of unauthorized activity and the application of updated enforcement rules. This metric is particularly relevant for scenarios involving lateral threat movement or real-time posture drift. Table VI summarizes comparative values. Our pipeline achieves a sub-300 ms reaction time—faster than all referenced frameworks—despite running on constrained hardware without centralized coordination. This demonstrates that combining static manufacturer-declared intent (MUD) with dynamic, SPA-driven posture checks yields superior enforcement agility, outperforming both SDN and blockchain-based architectures in responsiveness. Notably, our design also tolerates intermittent cloud connectivity and preserves enforcement integrity at the gateway level, unlike cloud-synchronized ZTA implementations.

H. Discussion on Adaptivity and Heterogeneous Validation

While MUD profiles are declarative and static by RFC 8520 design, our architecture introduces dynamic adaptability by combining SPA-triggered revalidation, DHCP lease-bound

policy expiry, and manufacturer-initiated profile updates. This extends traditional MUD enforcement beyond one-time admission control, aligning instead with Zero Trust principles of continuous verification. Unlike prior works that treat MUD as a static allowlist, our enforcement pipeline integrates runtime behavioral drift detection and posture-aware session continuity. This enables real-time reaction to policy changes without modifying the MUD specification itself. Moreover, our evaluation testbed was designed to reflect high heterogeneity in consumer IoT: ESP32-based microcontrollers (low-power), Linux-class Raspberry Pi nodes (mid-tier), and emulated traffic from devices like thermostats and IP cameras. This diversity in protocol patterns—spanning UDP telemetry, HTTPS polling, and persistent TCP connections—ensures the system’s validity across real-world usage profiles. Comparatively, most existing ZTA designs focus on homogeneous, high-power IoT or industrial deployments with stable communication patterns. Our system instead supports fluid, heterogeneous, intermittently connected environments typical of residential settings—without requiring centralized orchestration or heavy-weight agent-based monitoring. Future work will extend this validation to cloud-federated device fleets and Zigbee/Matter-enabled ecosystems to evaluate trust scoring convergence and adaptive enforcement scalability across vendor domains.

V. RESULTS AND DISCUSSION

The assessment confirms the capability of the suggested MUD–SDP architecture to apply the Zero Trust principles in consumer IoT networks without being a burden on the limited hardware. This part interprets performance metrics instead of just summarizing them as in Section IV and discusses their implications for practical deployment, QoS stability, adversarial resilience, operational efficiency, and regulatory readiness. Bounded jitter and stable round-trip latency from the viewpoint of service quality assure that the implementation of Zero Trust does not affect the overall responsiveness of real-time IoT applications. HVAC control loops, live camera monitoring, and telemedicine telemetry are very jitter-sensitive tasks where delay variation can be more disruptive than constant latency. The architecture kept the jitter below 4 ms during concurrent session load, meaning that per-flow enforcement via WireGuard and `iptables` can share the same resources with soft real-time traffic without infringing on QoS requirements.

The observations related to scalability add to the support of the deployment’s feasibility. Despite the presence of numerous simultaneous tunnels and policy checks that were actively taking place, the gateway built on Raspberry Pi 4B was still able to perform packet forwarding without experiencing any rule-evaluation stalls or creating a packet backlog. CPU was not allowed to use its full capacity, but still, it was enough for the usual number of smart home devices (20–30 active sessions). Moreover, in the case of denseness increasing like it would in a smart office or managed IoT hub, the separation of the SDP Controller would enable the transfer of the enforcement logic to the edge nodes that have better capabilities while the simplicity of the gateway is kept. The enforcement of security measures was accurate in both situations of policy

violations being standard and adversarial. The lateral flows that were not authorized were stopped and the deviations from the policy were restricted to 300ms thanks to the SPA-triggered revocation. The simulations of DNS spoofing and a rogue gateway further proved that the combination of intention-based enforcement with MAC-aware isolation is robust against impersonation and policy deception. The findings show that MUD-based behavioral whitelisting can provide a strong defense along with continuous SDP validation without the need of deep packet inspection or machine learning. The overall impact on administration is very important as well. The automation of the MUD-based policy derivation not only lowers the requirement for manual rule scripting but also solves one of the main barriers for adoption in consumer IoT environments with limited security experts. The auto-generation of per-device VLAN assignments, WireGuard tunnels, and firewall rules perfectly fits the secure-by-default deployment model and hence, minimizes errors in configuration. Operational visibility is made possible through structured logging that is compatible with SIEM pipelines. The gateway shares its rule decisions and violation events in formats that machines can parse, thus allowing centralized monitoring of different deployments and facilitating the coexistence of enforcement and telemetry in managed IoT security services. In addition, energy-aware enforcement considerations were investigated. Although periodic SPA keepalives ensure constant posture validation, they require the constrained devices to wake up, resulting in an overhead. The system provides adaptive keepalive tuning and event-driven SPA initiation, which makes it possible to balance the enforcement strength and the energy limitations for nodes powered by batteries. The architecture goes further than just implementing device-level control by way of identity-based access control for humans. The employed SPA-based authentication with optional MFA allows for the same Zero Trust logic applied to devices to selectively grant access to sensitive IoT resources, thus falling in line with NIST SP 800-207 guidance on identity-centric architectures. Moreover, the incorporation of a regulatory compliance matrix is clear evidence of the alignment with standards and the emerging mandates such as NIST SP 800-207, RFC 8520, RFC 9761, and the EU Cyber Resilience Act. It states clearly that the architecture is capable of not only technical enforcement but also auditability and compliance which are the character that real-world deployments expect. The overall outcomes strongly endorse the idea that the MUD–SDP integration has made Zero Trust in consumer IoT environments to be both practical and dynamic enforcement. The system continues to offer fine-grained segmentation and operational transparency by merging manufacturer intent with continual validation without the need for enterprise-class infrastructure or manual policy engineering.

VI. CONCLUSION AND FUTURE WORK

This paper presented a standards-aligned integration of Manufacturer Usage Description (MUD) policies with a Software-Defined Perimeter (SDP) enforcement framework to realize Zero Trust principles in consumer IoT environments. By combining manufacturer-declared behavioral intent

with SPA-driven session validation and WireGuard-based tunnel enforcement, the proposed architecture enables device-specific, least-privilege access control on resource-constrained gateways. Experimental evaluation on a Raspberry Pi-based testbed demonstrated that the system achieves low-latency secure communication while enforcing fine-grained segmentation using `iptables`, VLAN isolation, and per-session WireGuard tunnels. Under the defined threat model, unauthorized lateral movement, outbound policy violations, and network-layer spoofing attempts were successfully contained, with enforcement reactions occurring within 300 ms. MUD-driven automation further reduced manual configuration effort by over 75%, highlighting the practicality of deploying Zero Trust enforcement in consumer-grade environments. Several limitations remain. The current implementation relies on static MUD profiles for declarative intent, performs metadata-level inspection without deep application visibility, and bounds policy adaptation speed to manufacturer updates and DHCP lease cycles. In addition, while `iptables` ensures broad compatibility with embedded Linux gateways, `nftables` offers improved scalability and atomic policy management. Future work will focus on incorporating behavioral anomaly detection to refine MUD-derived policies dynamically, integrating `nftables` as the default enforcement backend, and evaluating multi-gateway deployments with distributed policy coordination in heterogeneous IoT ecosystems. These extensions will further strengthen secure-by-default Zero Trust enforcement aligned with emerging regulatory requirements. Overall, this work establishes a practical foundation for adaptive Zero Trust enforcement in consumer IoT environments by unifying manufacturer intent with context-aware, gateway-resident policy enforcement on constrained hardware.

VII. FUNDING

This research was funded by Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2026R234), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia.

REFERENCES

- [1] (2023). *Threat Landscape for Consumer IoT*. [Online]. Available: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2023>
- [2] P. Krishnan, K. Jain, K. Achuthan, and R. Buyya, "Software-defined security-by-contract for blockchain-enabled MUD-aware industrial IoT edge networks," *IEEE Trans. Ind. Informat.*, vol. 18, no. 10, pp. 7068–7076, Oct. 2022.
- [3] (2023). *Iot Security Report*. [Online]. Available: <https://www.paloaltonetworks.com/resources/research/2023-unit-42-attack-surface-threat-report>
- [4] F. Corno and L. Mannella, "A gateway-based MUD architecture to enhance smart home security," in *Proc. 8th Int. Conf. Smart Sustain. Technol. (SpliTech)*, Jun. 2023, pp. 1–6.
- [5] S. N. Matheu, J. L. Hernández-Ramos, S. Pérez, and A. F. Skarmeta, "Extending MUD profiles through an automated IoT security testing methodology," *IEEE Access*, vol. 7, pp. 149444–149463, 2019.
- [6] A. Poirrier, L. Cailleux, and T. Heide Clausen, "Is trust misplaced? A zero-trust survey," *Proc. IEEE*, vol. 113, no. 1, pp. 5–39, Jan. 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10970721/>
- [7] V. Morris and K. Kornegay, "Flow table modification using behavioral-based fingerprinting technique to facilitate zero trust identity management and access control," in *Proc. IEEE 15th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2025, pp. 307–314. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10903861/>
- [8] A. Iftikhar, F. B. Hussain, K. N. Qureshi, M. Shiraz, and M. Sookhak, "Securing edge based smart city networks with software defined networking and zero trust architecture," *J. Netw. Comput. Appl.*, vol. 244, Dec. 2025, Art. no. 104341.
- [9] D. Wing and B. Anderson. (2025). *Manufacturer Usage Description (MUD) for TLS and DTLS Profiles for Internet of Things (IoT) Devices*. [Online]. Available: <https://datatracker.ietf.org/doc/rfc9761/>
- [10] F. Lear, E. Lear, R. Droms, and D. Romascanu. (2019). *Manufacturer Usage Description Specification*. [Online]. Available: <https://tools.ietf.org/html/rfc8520>
- [11] S. Sankaran, S. Shivshankar, and K. Nimmy, "LHPUF: Lightweight hybrid PUF for enhanced security in Internet of Things," in *Proc. IEEE Int. Symp. Smart Electron. Syst. (iSES)*, Dec. 2018, pp. 275–278.
- [12] N. Sethu Subramanian, P. Krishnan, K. Jain, K. B. Aneesh Kumar, T. Pandey, and R. Buyya, "Blockchain and RL-based secured task offloading framework for software-defined 5G edge networks," *IEEE Access*, vol. 13, pp. 56820–56842, 2025.
- [13] M. Hammoudeh, D. Unal, A. Laouid, and F. Azzedin, "A blockchain-enabled zero-trust security architecture for securing Internet of Energy systems," *IEEE Netw.*, early access, Oct. 6, 2025, doi: [10.1109/MNET.2025.3613511](https://doi.org/10.1109/MNET.2025.3613511).
- [14] F. O. Minto, "Enhancing and testing smart home security through a mud-enabled environment," Ph.D. dissertation, Dept. Comput. Sci., Politecnico di Torino, Italy, 2023.
- [15] S. Datta, U. Venkanna, and K. M. Rao, "Mudscan: Double authentication based secure control mechanism for MUD enable IoT networks," *Peer-Peer Netw. Appl.*, vol. 18, no. 3, p.139, May 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s12083-025-01923-y>
- [16] N. Kaur et al., "Securing fog computing in healthcare with a zero-trust approach and blockchain," *EURASIP J. Wireless Commun. Netw.*, vol. 2025, no. 1, p.5, Feb. 2025. [Online]. Available: <https://link.springer.com/article/10.1186/s13638-025-02431-6>
- [17] M. I. Majid, F. Y. Khuhawar, and K. Muhammad, "NFV and secure cognitive sdn for educational backbone network deployment: Cognitive-enabled routing in SDN," in *Spectrum and Power Allocation in Cognitive Radio Systems*. Hershey, PA, USA: IGI Global, 2024. [Online]. Available: <https://www.igi-global.com/chapter/nfv-and-secure-cognitive-sdn-for-educational-backbone-network-deployment/353251>
- [18] X. Wang, Q. Yuan, Y. Wang, J. Teng, and J. Tao, "Zero trust-driven collaborative intrusion detection in IoT: A continuous trust assessment approach," *IEEE Internet Comput.*, vol. 29, no. 4, pp. 37–47, Jul. 2025.
- [19] H. Byeon et al., "Zero trust management over consumer technology-based IoT edge node for SDN communication and control of cyber-physical systems," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 4849–4858, May 2025.
- [20] K. Zhang, J. Sun, Q. Feng, X. Huang, C. Jiang, and M. S. Hos-sain, "Joint optimization of fairness and energy efficiency in zero-trust federated learning for consumer Internet of Things: A lossy communication perspective," *IEEE Trans. Consum. Electron.*, vol. 71, no. 1, pp. 1156–1166, Feb. 2025.
- [21] H. Zhu, X. Xue, M. Xu, and B.-G. Kim, "Zero trust consumer IoT with robust federated learning over main-side blockchain," *IEEE Trans. Consum. Electron.*, vol. 71, no. 1, pp. 1180–1189, Feb. 2025.
- [22] U. Kumaran, T. Swetha, I. Sharma, R. Keerthana, V. Tejashwini, and G. G. Devarajan, "Enhancing cloud IoT security with blockchain and SDN," in *Proc. Int. Conf. Commun., Comput. Sci. Eng. (IC3SE)*, May 2024, pp. 973–978.
- [23] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, Zero Trust Architecture, Standard SP 800–207, 2020, doi: [10.6028/NIST.SP.800-207](https://doi.org/10.6028/NIST.SP.800-207).
- [24] C. S. Alliance. (2022). *Software Defined Perimeter Specification V2.0*. [Online]. Available: <https://cloudsecurityalliance.org/artifacts/software-defined-perimeter-zero-trust-specification-v2>
- [25] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2017. [Online]. Available: <https://www.wireguard.com/papers/wireguard.pdf>
- [26] R. Ward and B. Beyer, "BeyondCorp: A new approach to enterprise security," *login: Usenix Mag.*, vol. 39, no. 6, pp. 6–11, Dec. 2014. [Online]. Available: https://www.usenix.org/system/files/login/articles/login_dec14_02_ward.pdf
- [27] (2025). *OpenZiti Documentation & Comparisons*. Accessed: 2025. [Online]. Available: <https://netfoundry.io/docs/openziti/>